

Context and Motivation

4D Gaussian Splatting (4DGS) is a leading approach to dynamic scene reconstruction from video.

Fast and flexible $\Rightarrow$ but not optimized!

Existing methods usually optimize one axis: **Fidelity**, **FPS**, **Memory**, or **Train Time**.

The best? *We test the optimizations together*

Quality Speed Memory Stability

4D Gaussian Building Blocks

Each primitive is centered in space-time and stores opacity, shape, rotation, and color. The native representation is expressive, but compact variants trade quality for speed and size.

$G_i = (\mu_i, \Sigma_i, o_i, r_i, \overline{\text{SH}}_i), \quad \mu_i \in \mathbb{R}^4, \quad \Sigma_i \in \mathbb{R}^{4 \times 4}$

4D mean	μ_i	mean is the Gaussian center
Covariance	Σ_i	(sphere ellipsoid) =: Isotropy
Opacity	o_i	adjusts visibility / see-through
Color	SH_i	$\text{SH}(0) = \text{RGB} \rightsquigarrow \text{SH}(3)$

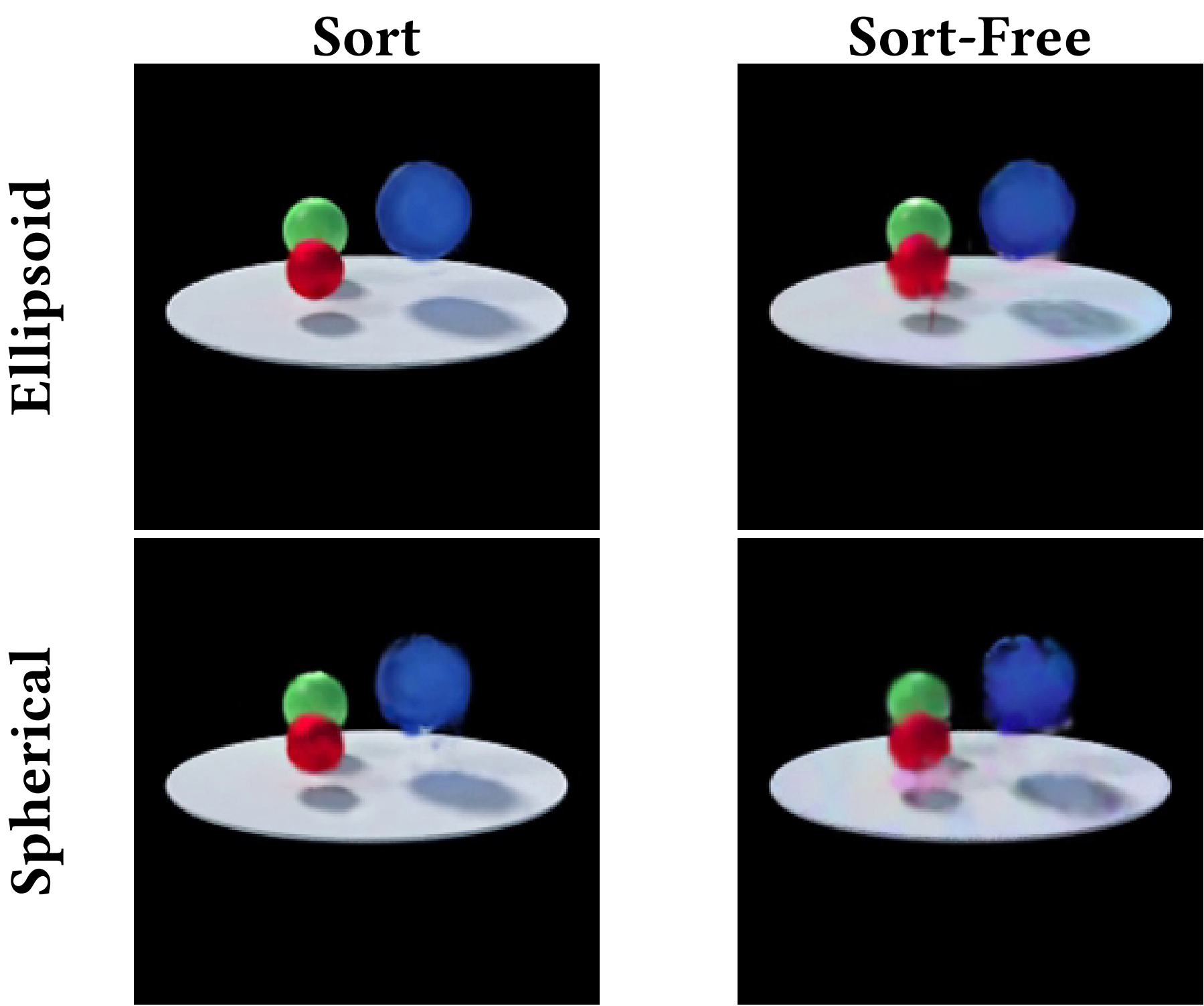
Ablations and Evaluation

Shape	Ellipsoid · Sphere
Color	RGB · SH(3)
Renderer	Sorting · Sort-Free
Pruning	Opacity · Spatio-Temporal
Densify	Edge-Guided, Gradient Based
Regularize	Dropout, Prune-Densify, Uncertainty

Dynamic D-NeRF Dataset: **T-Rex**, **Bouncing Balls**

Bouncing Balls

Fixed: No USplat/Prune/ESS/Dropout, Sort, 10k.



Bottlenecks Introduced

Sort-Free in 4D

Adding time to MobileGS
MLPs moves bottleneck from sorting to MLP eval.

Uncertainty loss

USplat-style weighting is too expensive for full ablations, so we tested small-scale.

Best Quality Ablation

ellipsoid · SH3 · sort · ESS · interleaved prune

	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	MB $\downarrow$	Gauss $\downarrow$
BBalls	4DGS	33.35	0.982	0.025	462	84	134k
BBalls	4DGS-1K	33.45	0.983	0.025	1509	13	20k
BBalls	Ours	34.38	0.985	0.017	1133	23	37k
TRex	4DGS	29.85	0.980	0.019	202	792	1265k
TRex	4DGS-1K	30.47	0.981	0.018	1361	118	189k
TRex	Ours	32.05	0.985	0.015	786	60	97k

Rendering Equation

Condition 4D distributions to time $t \Rightarrow$ 3D Gauss.
Project to camera plane and integrate colors.

Sort Depth-sort and composite front-to-back:

$$C_p^{\text{sort}}(t, v) = \sum_i \underbrace{T_i(p, t)}_{\text{leftover light}} \underbrace{\alpha_i(p, t)}_{\text{opacity}} \underbrace{c_i(v, t)}_{\text{color}} + \underbrace{T_{N+1}}_{\text{background}} c_{\text{bg}}$$

Sort-Free MLP-learned blending weights:

$$w_i(p, t, v) = \varphi_i^2(p, t, v) + \varphi_i \frac{p, t, v}{d_i^2(p, t, v)} + \exp\left(\frac{s_{\text{max}, i}}{d_i(p, t, v)}\right)$$

$$C_p^{\text{sf}}(t, v) = (1 - T_p(t, v)) \frac{\sum_i c_i(v, t) \alpha_i(p, t, v) w_i(p, t, v)}{\sum_i \alpha_i(p, t, v) w_i(p, t, v)} + T_p(t, v) c_{\text{bg}}$$

+ input time **t** $\rightarrow$ Works in 4D, but expensive MLP

Training Objective

By Ablation: fidelity, background, 4D motion.

Photometric Reconstruct each train view:

$$\mathcal{L}_{\text{rgb}} = \underbrace{(1 - \lambda_{\text{dssim}}) \mathcal{L}_1}_{\text{pixel fit}} + \underbrace{\lambda_{\text{dssim}} \mathcal{L}_{\text{SSIM}}}_{\text{structure}}$$

Opacity Do not learn the background:

$$\mathcal{L}_{\text{opa}} = -\frac{1}{|\Omega|} \sum_{p \in \Omega} \underbrace{(1 - m_{\text{gt}(p)})}_{\text{background mask}} \cdot \underbrace{\log(1 - \alpha(p))}_{\text{opacity penalty}}$$

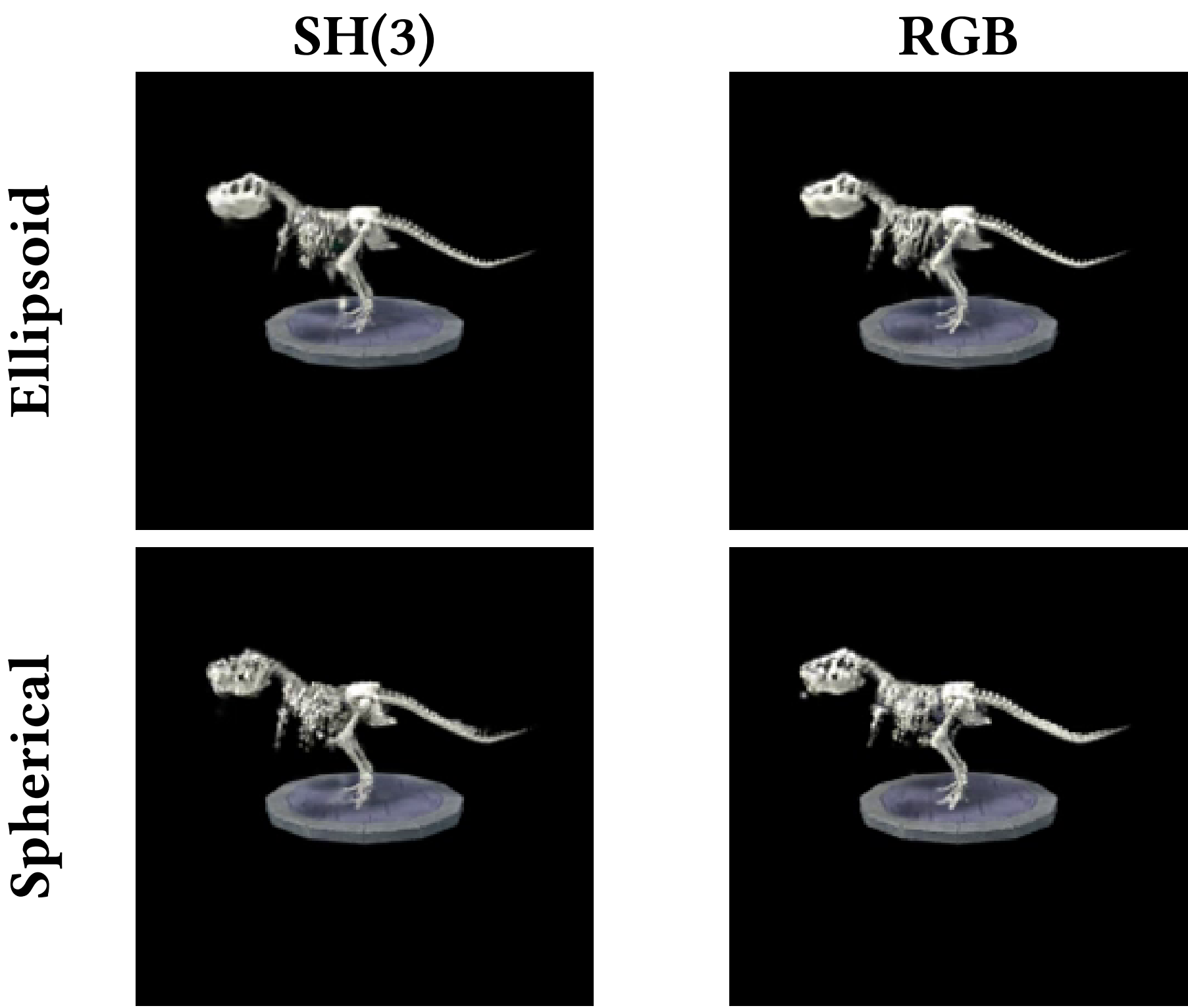
Motion Move softly (locally rigid, global speed):

$$\mathcal{L}_{\text{dyn}} = \underbrace{\mathcal{L}_{\text{rigid}}}_{\text{near Gaussians move similarly}} + \underbrace{\mathcal{L}_{\text{motion}}}_{\text{suppress high-velocity artifacts}}$$

+ **dynamic regularization** $\rightarrow$ better temporal consistency, but more expensive

TRex

Fixed: No USplat/Prune/ESS/Dropout, Sort, 20k.



Experimental Evaluation

Hedges' Coefficient $g > 0.5 \Rightarrow$ significant change

RGB +0.89	Smaller, lighter, faster; lower quality
SH(3) -0.89	Better quality, but expensive
Sort +0.84	Strong practical winner on our dataset
Sort-Free -0.84	Fewer Gaussians, but worse overall
	Checkpoint Train -93% VRAM -78% FPS -79% Gaussians -33% +79% SSIM +6.3%
	LPIPS -85% FPS +290% PSNR +29% SSIM +8.2% Train -85% VRAM -37%
	Gaussians -39% LPIPS +570% FPS -75% PSNR -23% SSIM -7.6% Train +590% VRAM +60%

Feature Matrix

existing		4DGS-Nat.	1000FPS	Instant4D	MobileGS	USplat4D	Omni-4DGS
heavily modified							
re-implemented							
Gaussians	Gaussians	4D 3D	●	●	●	●	●
	Rotation	Quaternion Rotation Matrix	●	●	●	●	●
	Shape	Isotropic Anisotropic	●	●	●	●	●
	Color Basis	RGB SH(1) SH(3)	●	●	●	●	●
Init	Point Cloud	Random MegaSAM	●	●	●	●	●
Train	Weighting	Uncertainty	●	●	●	●	●
	Sampling	Batch in Time	●	●	●	●	●
	Grid Reliance	Voxelization	●	●	●	●	●
Prune	Criterion	Contribution Gradient Loss	●	●	●	●	●
	Quantile Filter	Spatio-Temporal Opacity	●	●	●	●	●
	Strategy	One-shot Scheduled	●	●	●	●	●
	Increase	Gradient Based Edge-guided Split	●	●	●	●	●
Render	Dropout	Dropout	●	●	●	●	●
	Loading	Visibility Mask	●	●	●	●	●
	Raster	Sort-based Sort-free	●	●	●	●	●

MOG'D: The Bitter Lesson

Where is Tebe?



5 x moving Cameras

5 x still Cameras

Random Init $\Rightarrow$ Need *moving cameras* for artifacts.

Data and CUDA compatibility: the real bottlenecks.

Main takeaway

No configuration dominates all metrics

Practical Choice:

ellipsoid · RGB · sort-based · interleaved pruning · no dropout

Quality ellipsoid · SH(3) · sort-based rendering
Smallest RGB · interleaved pruning
Fastest sort-based in experiments (faster)

Single effect	Hedges' g	
	Smallest	RGB 4.91
	Eval VRAM	RGB 3.66
	PSNR	SH(3) 1.50
	LPIPS	Sort 1.46
	Render FPS	Sort 1.34
Training time	Sort	1.01

RGB $\Rightarrow$ -MB, -VRAM.
SH(3) $\Rightarrow$ +PSNR.
Sorted $\Rightarrow$ -LPIPS, +FPS, -train.

Bibliography

Kerbl et al. 3D Gaussian Splatting for Real-Time Radiance Field Rendering.
Yang et al. Native 4D Gaussian Splatting backbone.
Luo et al. Instant4D isotropic variants and Spatio-Temporal pruning.
Du et al. MobileGS Sort-Free rendering and compression ideas.
Hou et al. Sort-Free Gaussian Splatting.
Yuan et al. 4DGS at 1000 FPS visibility masks and pruning schedules.
Guo et al. Uncertainty-aware training for dynamic Gaussian splatting.